

보안 인프라 포트폴리오

남재근

구축한 환경이 실제로 동작하는지 끝까지 확인합니다.

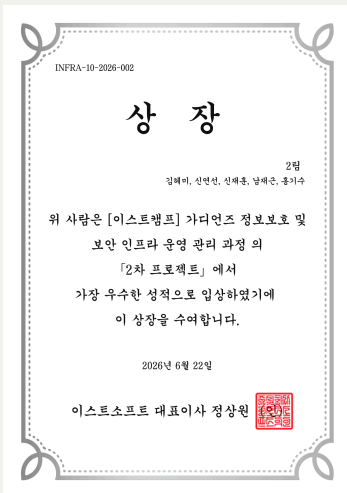
네트워크 구성, 접근통제, 웹 취약점 분석과 시스템 점검을 경험했습니다. 설정을 적용하는 데서 끝내지 않고 직접 시험하며 문제의 원인을 찾는 과정을 중요하게 생각합니다.

교육	가디언즈 정보보호 및 보안 인프라 운영 관리	프로젝트	팀 프로젝트 3회
수상	2차 프로젝트 우수상 · 창작 위게임 팀 1 위	Web	https://njgnet.github.io/

직접 참여한 프로젝트

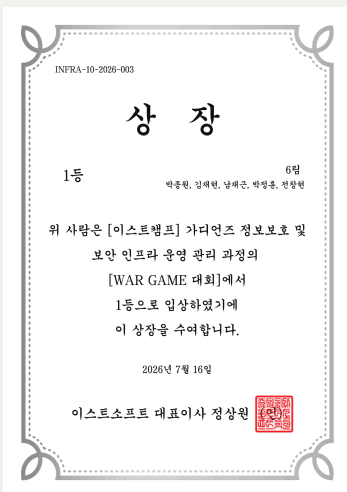
팀 프로젝트 3회

네트워크 구축 · 취약점 분석 및 시스템 점검 · 기업 정보보호 진단



2차 프로젝트 우수상

취약점 분석 · 시스템 점검 프로젝트



창작 위게임 팀 1위

Challenge 09-10 제작 및 전 문제 해결

문제를 구간별로 나누어 확인합니다.

예상과 다른 결과가 나오면 한 번에 여러 설정을 바꾸기보다 연결 상태, 적용 위치, 정책 순서와 서비스 조건을

SIMMIT PAY 네트워크 구축

Packet Tracer 기반 통합 인프라와 ACL 정책 검증

본사와 지사 네트워크를 연결하고 VLAN과 라우팅을 확인한 뒤 ACL을 적용했습니다. VPN과 Dynamic NAT는 팀원과 협업했으며, 저는 ACL 설정과 전체 통신 검증에 가장 집중했습니다.

핵심 담당

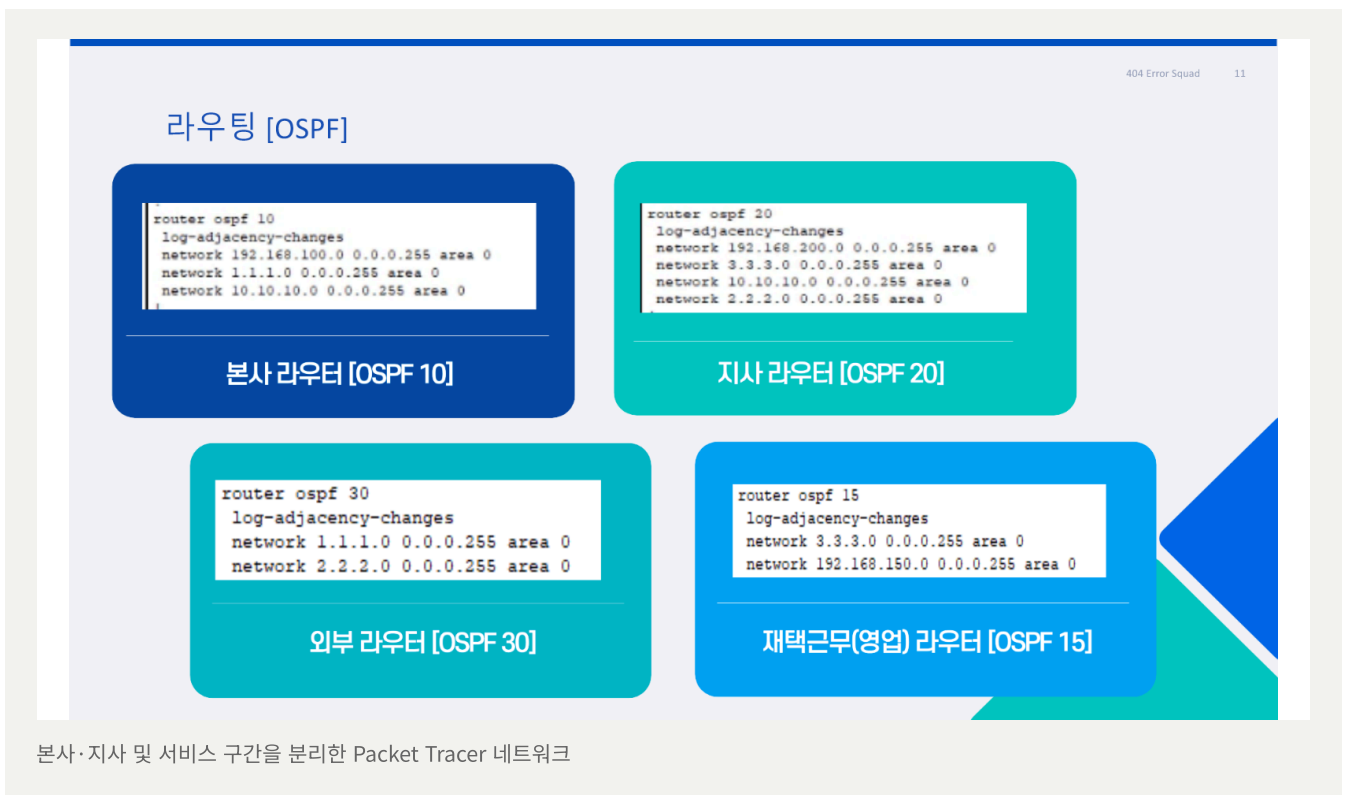
ACL 정책·통신 테스트

검증 범위

본사·지사·재택근무 3개 환경

증거 자료

본사·지사 시연 영상 2개



어려웠던 점

정책 한 줄로 정상 통신까지 차단됐습니다.

처음에는 통신이 되지 않을 때마다 ACL부터 수정해 원인을 구분하기 어려웠습니다.

통신 구간을 네 가지로 나누어 다시 확인했습니다.

01

본사 내부

부서별 WEB·FTP·DNS 사용 여부를 확인하고 DB와 LOG는 필요한 부서만 접근하도록 구분했습니다.

02

본사 → 지사

지사 WEB 접근과 필요한 서비스 통신은 허용하고, 임의의 Ping과 불필요한 부서 간 통신은 차단했습니다.

03

지사 → 본사

본사 WEB·DNS와 지사 서버의 LOG 전송을 확인하고 그 외 내부 구간 접근은 제한했습니다.

04

재택근무

업무에 필요한 WEB·FTP만 허용하고 내부망으로 직접 보내는 Ping은 차단했습니다.

반복 검증 과정

한 규칙을 수정해도 전체 시나리오를 처음부터 다시 실행했습니다.

1. ACL을 제외한 기본 라우팅과 인터페이스 확인
2. 출발지·목적지·서비스 기준을 한 항목씩 적용
3. 허용 대상의 성공과 차단 대상의 실패를 함께 확인
4. VPN·Dynamic NAT까지 포함한 최종 상태에서 본사·지사 시연 영상 제작

4구간

ACL 테스트 범위

3환경

본사·지사·재택근무

2개

직접 제작한 시연 영상

직접 맡은 부분과 협업한 부분을 구분했습니다.

핵심 담당

ACL 정책·검증

- 본사·지사 접근 기준 정리
- ACL 규칙과 적용 방향 설정
- 허용·차단 시나리오 반복 테스트
- 본사·지사 시연 영상 제작

구축 참여

VLAN·Routing

Packet Tracer에서 부서별 네트워크와 구간 연결을 구성하고 전체 통신 흐름을 확인했습니다.

팀 협업

VPN·Dynamic NAT

팀원과 함께 구성했으며, 팀원이 더 큰 비중으로 수행한 영역이어서 공동 작업으로 구분했습니다.

역할 분담

서버 연결 검증

서버 구축은 다른 팀원이 담당했습니다. 저는 네트워크 관점에서 필요한 통신이 서버까지 도달하는지 확인했습니다.

계획부터 최종 검증까지

계획서 작성·피드백	프로젝트 범위와 역할을 정리해 먼저 제출
네트워크 구성	VLAN과 OSPF Routing으로 본사·지사·외부 구간 연결
ACL 정책 적용	업무 요구사항을 부서·서비스별 허용·차단 규칙으로 변환
VPN·Dynamic NAT 협업	팀원이 중심이 되어 구성한 기능을 전체 환경에서 함께 확인
시연 및 최종보고	완성된 환경을 본사·지사 영상 2개와 PPT 보고서로 정리

정책 기준과 테스트 결과를 다시 확인할 수 있게 남겼습니다.

지사 구성

지사: 알의 자리수 1

지사 서버

지사 ACL 정책

- 본사 보안팀의 모든 접근 허용
- 모든 곳에서 WEB 사용 가능
- 지사 직원만 FTP 사용 가능
- 본사 로그 서버로 로그 정보 전송 허용
- 이외의 모든 접속 차단

본사·지사·재택근무 구간의 허용 및 차단 결과 확인

제출 및 증거 자료

- 프로젝트 계획서
- 1차 프로젝트 최종보고서
- 지사 네트워크 시연 영상
- ACL 정책 기준표
- 본사 네트워크 시연 영상
- Packet Tracer 네트워크 구성 자료

영상 제작 범위

전체 구성이 끝난 상태에서 통신 결과를 녹화했습니다.

ACL뿐 아니라 VPN과 Dynamic NAT까지 완료한 최종 환경에서 본사와 지사의 접근 허용·차단 및 서비스 통신을 시나리오별로 확인했습니다.

프로젝트를 마치고

설정 자체보다 원인을 나누어 확인하는 방법을 익혔습니다.

허용 테스트만으로 끝내지 않고 차단해야 하는 접근도 함께 시험했습니다. 서버 문제는 담당 팀원과 어느 구간에서 막혔는지 함께 확인했습니다.

ACL

부서·서비스별 최소 권한

2개

본사·지사 시연 영상

3환경

본사·지사·재택근무

테스트 과정을 다시 확인할 수 있는 자료로 남겼습니다.

시연 영상 2개

본사·지사 통신 테스트 직접 제작

3개 환경

본사·지사·재택근무 통신 검증

ACL

부서·서비스별 최소 권한 적용

프로젝트를 마치고

ACL을 하기 전에는 명령어 형식만 맞으면 된다고 생각했습니다. 실제로 설정해 보니 먼저 누가 어느 서버의 어떤 서비스를 사용해야 하는지 정리해야 했습니다. 허용 테스트만 해서는 부족했고, 막혀야 하는 접근이 실제로 막히는지도 같이 봐야 했습니다.

정책 순서나 적용 방향 하나가 잘못되면 정상 통신까지 막혀 원인을 찾는 데 시간이 걸렸습니다. 그래서 본사 내부, 본사에서 지사, 지사에서 본사, 재택근무 구간으로 나눈 뒤 출발지와 목적지, 서비스와 인터페이스를 하나씩 확인했습니다. 한 규칙을 수정한 뒤에는 허용 대상과 차단 대상을 처음부터 다시 시험했습니다.

서버 구축은 제가 맡은 영역이 아니어서 세부 설정까지 설명하기는 어렵습니다. 대신 네트워크에서 각 서버까지 필요한 통신이 도달하는지 확인했고, 문제가 생기면 서버 담당 팀원과 어느 구간에서 막혔는지 같이 확인했습니다. VPN과 Dynamic NAT까지 완료된 최종 환경에서는 본사와 지사 시연 영상을 직접 제작했습니다. 이번 프로젝트에서 제가 가장 많이 익힌 부분은 ACL 명령어 자체보다 통신 문제를 한 구간씩 나누어 확인하는 방법이었습니다.

Easy Hotel 취약점 분석과 시스템 점검

6건

웹 취약점 재현·조치

9 → 0

조치 전후 취약 항목

67개

시스템 점검 항목

64장

최종 통합보고서

2. 모의해킹 결과

2.1. 총평

본 시스템에 대한 보안 진단 결과, 총 6건의 주요 취약점이 식별되었습니다. 진단 항목은 크게 '인증', '회원 관리', '시스템 접근' 영역으로 구분되며, 각 항목에서 발견된 기술적 결함들은 시스템의 전반적인 보안 무결성과 데이터 보호 체계에 중대한 위험 요소로 작용하고 있습니다.

주요 진단 결과:

- 인증 및 세션 제어 취약점 (3.1.1 ~ 3.1.3):**
SQL 인젝션을 통한 인증 우회 가능성이 확인되었으며, Open Redirection 및 세션 고정(Session Fixation) 취약점이 도출되었습니다. 이는 공격자가 정상적인 인증 절차를 회피하거나, 탈취한 세션 정보를 이용하여 타인으로 위장 접속할 수 있는 심각한 위험을 내포하고 있습니다.
- 회원 정보 관리 및 인가 제어 취약점 (3.2.1 ~ 3.2.2):**
저장형 XSS(Stored XSS) 취약점을 통해 악의적인 스크립트 실행 및 사용자 정보 탈취가 가능하며, 부적절한 인가(IDOR) 결함으로 인해 타인의 예약 정보를 열람하거나 임의로 조작할 수 있는 상황입니다. 이는 개인정보 보호 및 서비스 데이터 무결성에 치명적인 영향을 줄 수 있습니다.
- 시스템 접근 제어 취약점 (3.3.1):**
SSRF(Server-Side Request Forgery) 취약점이 식별되어, 외부에서 시스템 내부 자원으로 비인가 요청을 수행할 가능성이 확인되었습니다. 이를 통해 서버 내부 인프라에 대한 추가적인 공격 시도가 우려됩니다.

결론 및 제언:

현재 본 시스템은 인증 우회, 데이터 조작, 서버 내부 망 접근 등 핵심 보안의 핵심 요소들이 복합적으로 존재하는 상태입니다. 각 항목별 발견된 취약점은 단독으로도 시스템의 보안을 무력화할 수 있는 수준이므로, 개별 단계에서의 보안 코딩 격증 및 검증 절차를 통해 발견된 모든 취약점에 대한 즉각적인 조치가 요구됩니다.

2.2 결과 요약

No	대상	취약점	위치
1		SQL 인젝션	http://easyhotel.com/member_login.php
2	로그인 페이지	Open Redirection	http://easyhotel.com/member_login.php
3		세션 고정	http://easyhotel.com/member_login.php
4	회원가입 페이지	Stored XSS	http://easyhotel.com/member_register.php
5	예약 상세 페이지	IDOR	http://easyhotel.com/member_reservation_detail.php
6	메인 페이지	SSRF	https://easyhotel.com/main.php

모의해킹 결과보고서에 정리한 6개 취약점

취약점 검증

공격 조건을 만들고 조치 후 같은 방법으로 재시험했습니다.

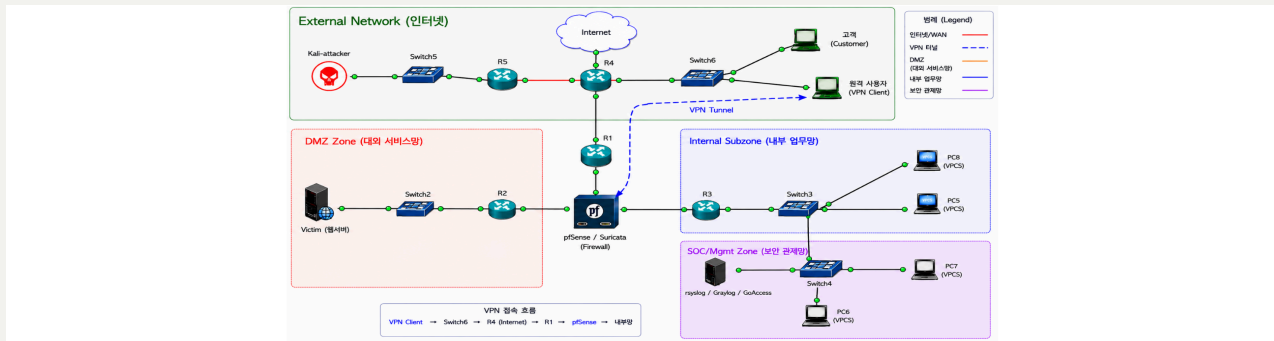
취약 코드 초안을 실제 환경에 적용한 뒤 공격이 재현되는 이유를 확인하고, 코드를 수정한 후 같은 입력으로 다시 시험해 차단 여부를 확인했습니다.

문서 작성

모의해킹 결과와 시스템 점검 내용을 정리했습니다.

취약점의 원인, 재현 조건과 조치 결과를 모의해킹 결과보고서에 정리하고 시스템 점검·최종보고서 작성에도 참여했습니다.

호텔 서비스의 공격부터 관제까지 연결했습니다.



외부망·DMZ·내부망·관제망으로 나눈 GNS3 환경

계획서 제출 후 피드백을 반영해 진행했습니다.

계획서 제출	호텔 예약 서비스를 기준으로 구축·공격·점검 범위 설정
환경 구성	GNS3 네트워크, pfSense·Suricata, 웹·DB와 로그 환경을 역할별로 구축
취약점 재현·조치	6개 웹 취약점의 공격 조건을 확인하고 코드 수정 후 재시험
시스템 점검	67개 항목 공통 틀과 집계 방식 구성, 조치 전후 결과 비교
보고서 제출	모의해킹·시스템 점검·셀 스크립트·64장 통합보고서 정리

제가 더 집중한 부분

전체 네트워크 구성에 참여한 뒤 취약점 재현과 조치 확인, 모의해킹 결과보고서 작성, 시스템 점검 스크립트 틀 작성에 더 많은 시간을 사용했습니다. pfSense·Suricata 정책과 웹·DB·관제 도구는 담당 팀원이 나누어 작업했습니다.

공격 화면보다 원인과 조치 후 결과를 함께 봤습니다.

01

SQL Injection

로그인 입력값과 쿼리 처리 방식을 확인하고 Prepared Statement와 비밀번호 검증 적용 후 같은 입력이 막히는지 재시험했습니다.

02

Open Redirection

로그인 이후 전달되는 이동 경로를 조작해 외부 주소로 이동하는지 확인하고 허용 경로 검증을 적용했습니다.

03

IDOR

예약 번호만 바꾸어 다른 사용자의 정보에 접근되는 조건을 확인하고 로그인 사용자와 예약 소유자 검증을 추가했습니다.

04

File Upload

확장자와 MIME 형식 검증이 부족한 업로드 경로를 시험하고 허용 형식과 저장 위치를 제한했습니다.

05

관리자 페이지 접근

일반 사용자가 관리자 기능에 접근할 수 있는 조건을 확인하고 서버 측 권한 검사를 보완했습니다.

06

Stored XSS

저장된 입력값이 다시 출력될 때 스크립트가 실행되는지 확인하고 출력 인코딩과 입력 검증을 적용했습니다.

보고서 구성

취약점 설명 → 취약한 코드 → 재현 절차 → 공격 증거 → 대응 방안 → 수정 코드 → 조치 후 결과의 순서로 정리했습니다. 공격 성공 여부만 보여주는 대신 왜 발생했고 무엇을 바꾼 뒤 막혔는지 이어서 확인할 수 있게 작성했습니다.

점검 스크립트의 틀을 먼저 시험했습니다.

01

어려웠던 점

67개 항목을 나누어 작성하기 전에 공통 출력 형식과 집계 기준이 필요했습니다.

02

초기 틀 제작

멘토님의 피드백을 바탕으로 시로 초안을 잡고 Ubuntu에서 명령어와 경로를 직접 확인했습니다.

03

팀 공유

U-01~U-67 실행 결과가 한 화면에 집계되는 형태를 캡처해 팀원들에게 먼저 보여줬습니다.

04

재검증

양호 58개·취약 9개에서 설정을 수정한 뒤 67개 전체 양호를 확인했습니다.

```
=====
점 검 요 약      전 체 : 67      양 호 : 양 호 58      취 약 : 취약 9 수 동 확 인 : 수 동 확 인 0
=====
```

조치 전 · 양호 58 / 취약 9

```
=====
점 검 요 약      전 체 : 67      양 호 : 양 호 67      취 약 : 취약 0 수 동 확 인 : 수 동 확 인 0
=====
```

조치 후 · 67개 전체 양호

제가 먼저 보여준 부분

점검 함수의 형태와 결과 집계 방식을 먼저 실행해 본 뒤 “이런 방식으로 합치면 될 것 같다”고 공유했습니다. 각 팀원이 작성한 항목을 같은 형식에 넣을 수 있도록 공통 출력 함수와 실행 순서를 정리했습니다.

채택 여부와 별개로 직접 동작까지 확인했습니다.

CRON · TTY OUTPUT TEST

예약 실행 결과를 활성화된 Ubuntu 터미널에서 바로 확인하는 방식

점검 결과를 로그 파일에 저장하는 것에 더해, 관리자가 별도 명령어를 입력하지 않아도 현재 열려 있는 터미널에서 결과를 확인할 수 있으면 좋겠다고 생각했습니다.

실행 시간을 지정하고 활성 터미널로 결과를 보내는 방법을 직접 시험했습니다. 팀의 최종 운영 방식으로 채택되지는 않았지만, 화면에 결과가 나타나는 과정까지 확인해 시연 영상으로 남겼습니다.

[시연 영상 열기 ↗](#)

2.2 보안 점검 자동화 및 상시 모니터링 체계

보안 점검 자동화 및 상시 모니터링 체계 구축

본 시스템은 보안 점검의 누락을 방지하고 일관된 보안 수준을 유지하기 위해, 자동화 스크립트를 활용한 정기 점검 체계를 구축하였습니다.

- **자동화 구현:** 리눅스의 cron 서비스를 활용하여 매일 오후 1시에 보안 점검 스크립트(security_check.sh)가 자동으로 실행되도록 설정하였습니다.
- **설정 이유:** 본 시스템은 호텔이 체크인(15:00) 및 체크아웃(11:00) 업무가 최소화되는 매일 오후 1시를 보안 점검 자동화 시간으로 설정하였습니다. 이는 서비스 가용성을 최우선으로 고려한 조치이며, 점검 결과를 당일 업무 시간 내에 확인하여 신속한 사후 조치가 가능하도록 운영 효율성을 극대화하였습니다.
- **상시 기록 체계:** 점검 결과는 /var/log/security_report_[날짜].log 파일로 자동 저장되며, 이를 통해 관리자가 별도의 수동 작업 없이도 시스템의 보안 상태를 상시로 확인하고 추적할 수 있도록 하였습니다.
- **보안 유지:** 매일 수행되는 자동 점검을 통해 기존에 조치 완료한 67개 항목이 '양호' 상태로 지속 유지되고 있음을 검증하고 있으며, 이상 징후 발생 시 즉각적으로 인지할 수 있는 운영 체계를 확립하였습니다.

2.3 보안 자동화의 전략적 이점 및 설계 원칙

본 자동화 체계는 단순한 효율을 넘어, 시스템 운영의 안정성과 보안 무결성을 최우선으로 고려하여 설계되었습니다.

1) 자동화 도입 효과 및 사용자 이점

- **운영 효율성 극대화:** 기존의 수동 점검 방식(항목당 5~10분 소요, 휴먼 에러 발생 가능성)을 배제하고, Cron을 통한 자동 실행으로 리소스 소모를 99% 이상 절감하였습니다.
- **상시 가시성 확보:** 점검 시점 외에도 설정 변경 사항을 즉시 감지하여 선제적 위험 대응이 가능합니다.
- **컴플라이언스 대응 최적화:** KISA 가이드라인 기반의 결과 보고서를 즉시 생성하여, 향후 외부 보안 컨설팅 및 정기 점검 시 대응 준비 시간을 대폭 단축했습니다.

2) 셸 스크립트(Shell Script) 채택의 보안 전략

보안 자동화 도구 자체가 새로운 취약점이 되는 상황을 방지하기 위해 다음과 같은 원칙을 적용하였습니다.

- **최소 권한 및 순정 가능 활용:** 파이썬 등 외부 프로그램 설치로 인한 '배경 프로세스 취약점' 발생을 원천 차단하기 위해, 리눅스 순정 가능한 셸 스크립트만을 사용하였습니다.
- **검증 가능성 및 유연성:** 코드가 직관적이며서 보안 담당자가 즉시 악성 코드 여부를 검증할 수 있으며, 새로운 취약점 발견 시 즉각적인 코드 수정과 반영이 가능합니다.
- **스크립트 파일 무결성 관리:** 스크립트 자체의 보안을 위해 파일 권한을 chmod 700(또는 600)으로 설정하여, 관리자 외 비인가자의 접근을 원천 차단하였습니다.

제가 맡은 작업과 팀의 작업을 구분해 정리했습니다.

가장 많이 한 부분

취약점 분석·재현

- 공격 조건과 입력값 테스트
- 보안 조치 후 같은 공격 재시험
- 모의해킹 결과보고서 작성 참여

직접 작성

점검 스크립트 틀

67개 함수가 같은 형식으로 출력되도록 공통 함수, 결과 집계와 실행 순서를 먼저 구성했습니다.

제안·시연

cron 터미널 출력

예약 실행 결과를 활성 Ubuntu 터미널에 보여주는 방식을 시험하고 시연 영상으로 남겼습니다.

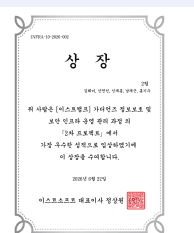
문서 작성 참여

결과보고서 정리

모의해킹, 시스템 점검과 64장 최종 통합보고서 작성 과정에 참여했습니다.

최종 제출 자료

- 프로젝트 계획서
- 시스템 점검보고서
- 64장 최종 통합보고서
- 모의해킹 결과보고서
- 시스템 점검 셸 스크립트
- cron 터미널 출력 테스트 영상



RESULT

2차 프로젝트 1위 · 우수상

취약점 재현과 보안 조치, 시스템 점검 및 문서 결과를 팀원들과 함께 완성해 우수상을 받았습니다.

점검 스크립트의 틀부터 실제 동작까지 먼저 시험했습니다.

67개 항목

공통 출력 형식과 결과 집계 구성

조치 전·후

같은 스크립트로 결과 재검증

시연 영상

cron 터미널 출력 방식 직접 시험

프로젝트를 마치고

1차에서는 네트워크 연결과 ACL에 집중했다면 2차에서는 웹 코드와 Ubuntu 시스템 설정까지 같이 봤습니다. 취약 코드 초안을 실제 환경에 적용한 뒤 왜 공격이 되는지 분석하고, 코드를 수정한 다음 같은 입력으로 다시 시험해 차단되는지 확인한 과정이 기억에 남았습니다.

점검 스크립트 작업에서는 67개 항목을 각각 작성하는 것보다 팀원들이 만든 결과를 같은 형식으로 합칠 수 있는 공통 틀이 먼저 필요했습니다. 멘토님의 피드백을 받은 뒤 출력 형식과 결과 집계 방식을 먼저 시험했습니다. U-01부터 U-67까지 실행 결과와 양호·취약 개수가 한 화면에 보이는 초기 형태를 만든 다음 결과 화면을 캡처해 팀원들에게 보여주며 “이런 방식으로 합치면 될 것 같다”고 작업 방향을 제안했습니다.

예약된 시간에 점검 스크립트를 실행하고 결과를 활성화된 Ubuntu 터미널에서 바로 확인하는 방식도 제안했습니다. 최종 운영 방식으로 채택되지는 않았지만 아이디어로만 끝내지 않고 실제 화면에서 동작하는지 확인해 시연 영상으로 남겼습니다. 설정을 수정한 뒤에는 같은 스크립트를 다시 실행해 조치 전 양호 58개·취약 9개에서 67개 전체 양호로 바뀌는 것도 확인했습니다. 이 과정에서 결과만 완성하는 것보다 막힌 부분을 어떤 순서로 확인하고 해결할지가 중요하다는 것을 배웠습니다.

PrivaDrive 기업 정보보호 진단

7종

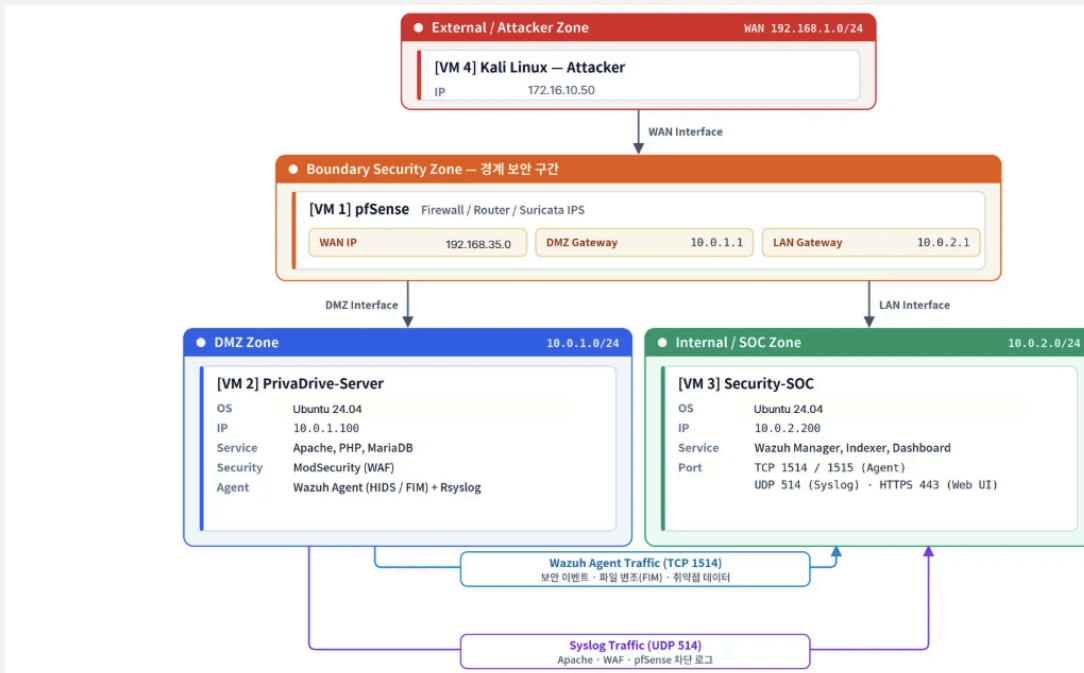
웹 취약점 점검

4종

악성파일 분석 도구

56장

직접 작성한 발표자료



External·Boundary·DMZ·SOC 구간으로 구성된 기업 인프라

취약점 조치

공격 결과와 발생 조건을 확인하고 조치 과정에 참여했습니다.

애플리케이션 코드와 보안 설정에서 수정할 부분을 정리하고, 조치 후 재시험 결과를 확인했습니다.

발표자료 단독 작성

팀 산출물을 56장 발표 흐름으로 다시 구성했습니다.

계획서, 구성도, 모의해킹, 탐지·차단, 로그와 악성코드 분석 결과를 목적→점검→발견→조치 순서로 정리했습니다.

공격·조치·탐지 결과가 한 흐름으로 이어지게 정리했습니다.

01

취약점 조치

7개 웹 공격의 입력값과 실제 영향을 확인하고 애플리케이션 코드와 보안 설정에서 수정할 부분을 정리했습니다.

02

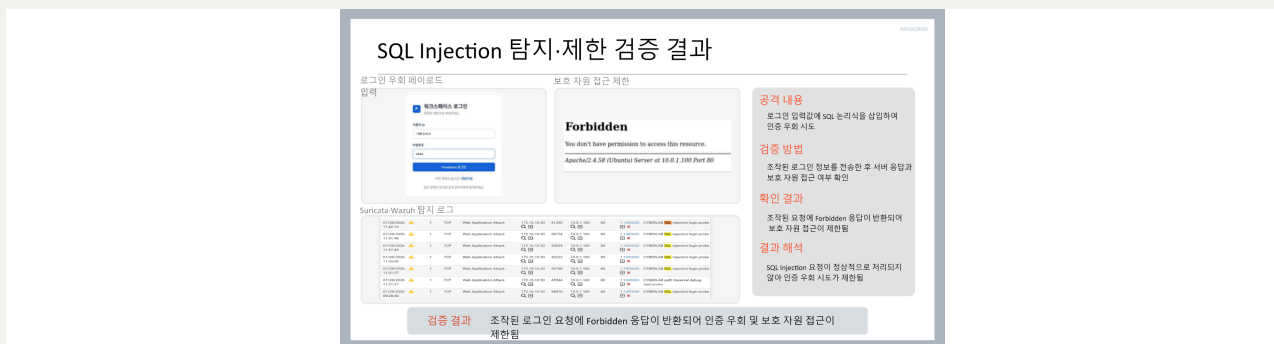
탐지와 차단 확인

같은 공격이 WAF·IDS·Wazuh에서 어떻게 보이는지 확인하고 조치 후 재시험 결과를 비교했습니다.

03

발표 흐름 구성

계획서부터 구성도, 공격, 탐지·차단, 로그, 악성파일 분석과 셸 스크립트까지 56장으로 다시 구성했습니다.



SQL Injection의 공격 결과, 애플리케이션 조치와 관계 확인을 한 화면에 정리

어려웠던 점

팀원별 결과를 나열하는 것만으로는 흐름이 보이지 않았습니다.

각 화면에서 무엇을 시도했고 어느 계층에서 막혔으며 어떤 로그가 남았는지를 다시 확인해야 했습니다.

정리 방식

목적 → 점검 → 발견 → 조치 → 재시험 순서를 맞췄습니다.

기술 구현은 팀 협업 결과로 구분하고, 제가 직접 맡은 취약점 조치 참여와 발표자료 전체 구성은 분명하게 표시했습니다.

팀의 결과를 발표 가능한 한 흐름으로 다시 구성했습니다.

직접 맡은 부분

취약점 조치 참여 · 56장 발표자료 전체 작성

모의해킹 결과에서 발생 조건과 영향을 확인하고 조치 내용을 정리했습니다. 이후 계획서, 구성도, 공격·조치, 탐지·차단, Wazuh 로그, 악성파일 분석과 셸 스크립트 결과를 발표 순서로 다시 구성했습니다.

초기 웹 서버 코드는 팀원이 만든 참고안에서 시작했으며, 시스템 시연 영상 역시 팀원이 제작한 자료입니다. 해당 부분은 제 개인 구현 성과로 표현하지 않았습니다.

팀 역할표 · 남재근: 취약점 조치

최종 제출 자료

- 3차 프로젝트 최종 계획서
- 모의해킹 결과보고서 45P
- 기업정보보호 결과보고서 21P
- 발표자료 56P
- 네트워크 구성도
- 악성코드 분석 결과보고서
- 자동 차단 셸 스크립트
- 팀 제작 시연 영상

3차 프로젝트 PrivaDrive

박정훈, 박종원, 전창현, 남재근, 김재현

발표자료 | 3차 프로젝트 | DATE: 07.27~08.05 | Status: 4월

3차 프로젝트 발표자료

프로젝트 전체 과정을 56장으로 직접 구성

팀원별 결과를 하나의 흐름으로 다시 구성했습니다.

취약점 조치

공격 조건과 조치 후 결과 확인

전체 흐름

탐지·차단·로그 결과 연결

발표자료 56장

프로젝트 전체 내용 직접 구성

프로젝트를 마치고

3차에서는 앞선 프로젝트에서 다뤘던 네트워크, 웹 취약점과 시스템 점검이 하나의 환경 안에서 연결됐습니다. 같은 공격이라도 애플리케이션 코드, WAF, IDS와 관제 화면에서 보이는 정보가 달랐습니다. 팀원들이 맡은 화면과 로그를 그대로 나열하는 것만으로는 무엇을 시도했고 어느 단계에서 탐지되거나 차단됐는지 설명하기 어려웠습니다.

발표자료 전체를 맡아 작성하면서 각 결과의 공격 조건과 조치 내용을 다시 확인했습니다. 제가 직접 맡지 않은 작업은 팀원에게 질문하고 관련 보고서와 화면을 살펴보며 애플리케이션, 보안 장비와 관제 로그가 어떻게 이어지는지 파악했습니다. 이후 프로젝트 목적, 점검, 발견 사항, 조치와 재시험 순서로 내용을 맞췄습니다.

기술 이름을 많이 넣기보다 어떤 문제가 있었고 무엇을 수정했으며, 같은 조건으로 다시 시험했을 때 결과가 어떻게 달라졌는지를 보여주는 데 집중했습니다. 계획서부터 네트워크 구성도, 취약점, 탐지·차단, Wazuh 로그, 악성파일 분석과 셸 스크립트까지 팀 산출물을 56장 발표자료로 구성했습니다. 이 과정에서 제가 맡은 부분뿐 아니라 각 결과가 서로 어떻게 연결되는지도 이해해야 전체 프로젝트를 설명할 수 있다는 점을 배웠습니다.

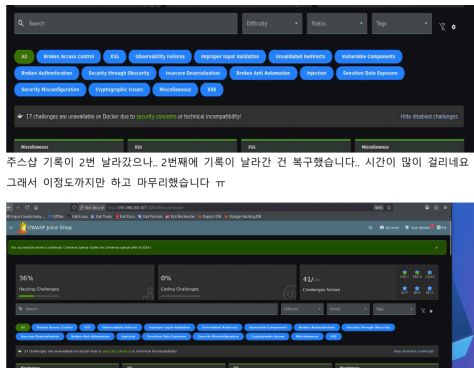
문제를 만들고 직접 풀어봤습니다.



TEAM CTF

문제 출제 의도와 워크스루 작성

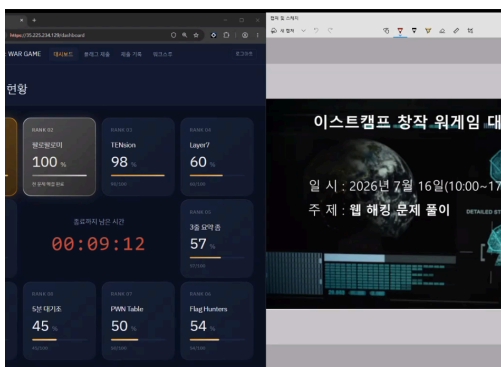
2팀 EasyHajo로 참가해 14개 플래그를 획득하고 팀 5위를 기록했습니다.



OWASP JUICE SHOP

요청값이 서버에서 처리되는 과정을 확인

관리자 인증 우회, 별점 요청값 변조와 관리자 권한 필드 추가를 실습하고 원인과 대책을 보고서로 작성했습니다.

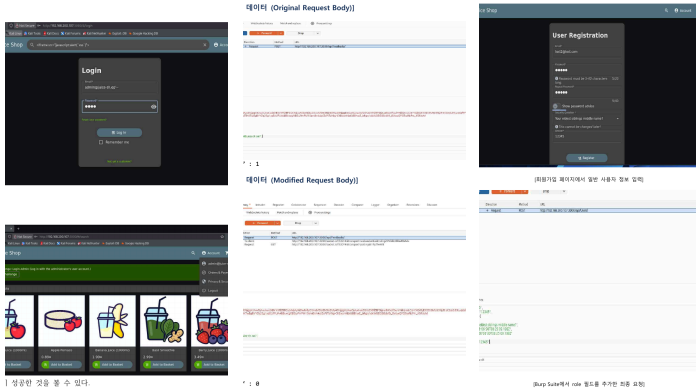


CREATIVE WARGAME

Challenge 09·10 제작 · 팀 1위

보안 설정 오류와 Typosquatting 공급망 공격 문제를 제작했으며, ::6 팀으로 전체 10개 문제를 해결했습니다.

풀이 결과뿐 아니라 과정과 자료를 함께 남겼습니다.



OWASP JUICE SHOP

관리자 인증 우회·별점 변조·권한 필드 추가

브라우저 화면만 보는 대신 요청값이 서버에서 어떻게 처리되는지 확인하고, 성공 조건과 필요한 보안 대책을 보고서로 작성했습니다.

취약점 문제 유형

Challenge 번호	담당자	문제 유형
01	박종원	Authentication Bypass (SQL Injection)
02	박종원	Reflected XSS (xss)
03	박종원	Source Map Exposure / Build Artifact Information Disclosure (소스맵 노출 및 빌드 산출물 정보노출)
04	박종원	XML External Entity Injection (XML 태그 속성 엔티티를 주입 변조)
05	전창현	Search Center SQL Injection 취약점 검색 기능의 쿼리 처리 오류
06	전창현	Network Check 명령어 삽입 취약점 진단기능 입력값 검증 미흡
07	박정훈	브라우저 저장소(Storage) API를 직접 건드리는 문제
08	박정훈	SSRF(Server-Side Request Forgery)(외부 노출용 웹 소스를 통해서 내부만 전용 디렉토리에 접근)
09	남재근	Deprecated Interface (보안 설정 오류(Security Misconfiguration))
10	남재근	Legacy Typosquatting (공급망 공격(Supply Chain Attack))

팀원별 Challenge 01~10 문제 유형

CREATIVE WARGAME

Challenge 09·10 제작

09번은 오래된 인터페이스에 따른 보안 설정 오류, 10번은 Typosquatting 공급망 공격을 주제로 구성했습니다. ::6 팀으로 전체 문제를 해결해 1위를 기록했습니다.

CTF 기록

2팀 EasyHajo로 CTF에 참가해 14개 플래그를 획득하고 5위를 기록했습니다. 팀 문제의 출제 의도와 해결 절차는 별도의 워크스루 PDF로 정리했습니다.

배운 내용을 실습으로 확인했습니다.

01

네트워크·인프라

Linux, Cisco, VLAN, Routing, ACL, VPN, NAT, Windows Server

02

보안 운영·관제

pfSense, Suricata, Snort, Wazuh, 로그 분석과 시스템 점검

03

웹 취약점 실습

DVWA, WebGoat, OWASP Juice Shop, Metasploit, WAF

04

문제 해결 방식

환경 구성 → 결과 확인 → 원인 분리 → 조치 → 같은 조건으로 재시험

PORTFOLIO LINKS

상세 화면과 원본 자료는 웹 포트폴리오에서 확인할 수 있습니다.

<https://njgnet.github.io/>

<https://github.com/njgnet>

PDF 출력 안내

이 문서는 웹 포트폴리오의 주요 경험과 근거 자료를 A4에 맞게 재구성한 출력본입니다. 프로젝트 원본 보고서와 시연 영상은 각 상세 페이지의 링크에서 확인할 수 있습니다.